AI-Based Log File Compression for Cloud Systems

Ramya B, Ameen Basha M, and Dr. Karthik K
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology (VIT)
Vellore, Tamil Nadu, India
ramya.b2023@vitstudent.ac.in, ameenbasha.m2023@vitstudent.ac.in, k.karthik@vit.ac.in

***Abstract*—Cloud platforms collect copious amounts of data, such as textual application logs, server logs and structured cloud activity records, which is directly related to usage-based storage costs. This footprint can be lower with lossless compression, but the optimum compression algorithm will vary based on the file type, structure, and compression speed, memory usage and runtime. Studies of general-purpose codecs, studies of log-specific compressors, and studies of learned compressors are more or less distinct, and the use of an upload file, select a compression method, check reconstruction, and evaluate storage impact has been carried out for these types of codecs, but they were generally studied independently of one another. It proposes a cloud-based File-Compression Management System which enables the uploading of the files, checks the uploaded files, identifies types of files, classifies the compressed file as Cloud Log Files and Cloud Activity Files, applies proper preprocessing, compresses and measures file size and file compression duration, provides results dashboard, decompresses and verifies file integrity (optional) and compares cost estimation to store the compressed file. All of the measures of the evaluation (compression ratio, space saved, compression and decompression throughput, round trip integrity, estimated storage-cost difference) are reported in the paper and a documented experimental protocol is described. It's organised based on the type of organisations (6 categories) not live cloud integrations like AWS EC2, AWS Lambda, AWS S3, Azure Virtual Machines, Google Cloud Compute Engine and Cloud Application Servers. The compression rate and the decompression rate of the dashboard in the prototype used are assumed/simulated and it is assumed that the verification and cost values of the dashboard are known, and the performance of the codec is performed at the endpoint separated from the dashboard – the benchmark endpoint. All 216 combinations of compression methods GZIP, Zstandard and LZMA and compression level were the same compressed identical number of bytes back from the original in all 16 public Loghub logs and 8 synthetic activity files tested; aggressive compression ratio ranges varied from 7.63 to 13.60 bytes back from the original in the 16 logs, and 4.94 to 7.23 bytes back from the original in the 8 synthetic activity files; in the case of logs, Zstandard level 3 compressed over 13 times faster at a similar ratio, as GZIP level 9. These measurements are taken on the above mentioned data and are not an end-to-end measurement of the web application, nor is the AI Analysis component measured.**

***Index Terms*—Sniffing the lossless data, cloud logs, cloud activity Records, Storage optimization, Log file compression, Integrity verification, compression benchmarking, file-type detection.**

I. Introduction

Today it is commonplace for organizations to gather telemetry information from virtual machines, serverless functions, object stores, and application servers via cloud computing [29], [30]. This operational data is all continuously generated: Application and server log activity will always be semi-structured text, and cloud activity exports will always be structured rows or nested documents, containing API calls, resource changes, and access events. Much of this data needs to be retained over long periods, as advised by the log management guidelines [31] for troubleshooting, auditing and compliance purposes. As the cloud usage of the object and the block storage services is typically charged by the amount of data stored, as stated in the provider pricing document (e.g., [15]), minimizing the number of bytes stored is a direct lever on one of the cloud bill.

Lossless compression is the natural mechanism, since logs Replication of and audit records shall be as accurate as possible. Mature general- There are methods available such as purpose methods based on DEFLATE. GZIP [3], [4], Zstandard [5], Brotli [6], and LZMA-based All of the above are XZ, based on dictionary and entropy-coding. principles introduced in [1], [2] and refined in [7], [8]. These The different methods have different compression ratios, speed and memory requirements, The relationships and their relative behavior depend on the data. Highly repet- Templated messages are different from Granted, timestamp-heavy tabular exports or from already-compressed binary files. Experimentation with exploiting is widespread among logs. It has been found that a large portion of logs have experimented with exploiting. Using message templates and structure can have a beneficial impact on results beyond. general-purpose tools [10]–[12], [23], while neural approaches Investigate more complicated models that are more expensive to compute [14].

There are two types of operational files: Cloud Log Files are textual logs like application logs or server logs, where each line is a loosely structured message that includes a timestamp, a level, and freely structured text content; Cloud Activity Files are structured cloud activity or event records, usually in the form of structured data exports (e.g., CSV, JSON, etc.), where the fields and schema are repeated across each record. The redundancy pattern for the two categories is different, and may require different choices of preprocessing and codecs. This distinction, however, would be lost in all records of activity, and so the two types are maintained separate for purposes of this paper.

However, if one only wants to safely reduce files for some reason, there are a few problems to work around in practice. First, the selection of a method is not trivial as it is possible that a high compression ratio at a very high runtime or memory

cost may not be appropriate. Second, if the original can be recovered exactly, then compression is useful only if it is done explicitly followed by an integrity check [9]. Third, savings claims can only have meaning if they are evaluated within a given protocol with repeatable runs and if storage-cost estimates include pricing assumptions. Fourth, the way files are handled in real deployment is diverse; a system needs to check file contents and identify the kinds of files before taking any action based on the file category.

The research gap I address here is not algorithmic, as there are good individual compressors and log-specific techniques in the literature; it is practical and integrative, as the works referenced in this paper focus on algorithms, log/activity types, applicable preprocessing, measured compression, verified decompression and the estimation of storage cost, and none of them combine these components into a user-facing workflow that includes a documented evaluation protocol. The idea of a system does not exist does not mean it is not possible, this paper did not conduct a systematic literature search for this.

The main contributions of this work are as follows:

- A categorized compression-management workflow: upload, validation, detection of the type of the uploaded file, categorize into Cloud Log Files and Cloud Activity files, apply applicable preprocessing, compression, measure file size and time, report to a dashboard, decompression, and optional integrity verification (in the prototype, only the benchmark endpoint acts on the real file bytes; Section III).
- An evaluation framework for the formal method: compression ratio, percentage of space saved, compression time/throughput, decompression time/throughput, round trip integrity, and estimated storage-cost difference (units and assumptions).
- A documented experimental protocol with repeated run measurements, per category benchmarking, and statistical reporting.
- A measured benchmark for codecs: compression ratio, throughput, decompression speed, verify round trip, and illustration of storage cost for GZIP, Zstandard, and LZMA on public log samples and synthetic activity files.
- Transparent scope of statements – The six environment categories are interface level organizational categories, not live integrations, and the AI Analysis component is described in as much detail as confirmed functionality.

The rest of this paper is structured as follows: Section II reviews related work. The system architecture, the preprocessing, the compression, the verification, the cost estimation and the mathematical formulation are described in Section III. The experimental setup and the structure of the results are given in Section IV. Trade-offs and limitations are discussed in Section V and future work is outlined in Section VI.

II. Literature Survey

A. Lossless Compression Algorithms

The first idea behind dictionary-based compression is the sliding-window idea of Ziv-Lempel [1] which was developed after Shannon's information theory [19] was introduced. A few later techniques are: the variable-rate parsing [20], LZW [21], and arithmetic coding [22]. DEFLATE is the basis of GZIP [4] and is a combination of LZ77 matching and Huffman coding [3]. Zstandard implements dictionary compression and a number of compression levels [5] with asymmetric numeral systems for entropy coding [8]. Brotli uses LZ77, Huffman coding, context modelling and a static dictionary [6]. The Burrows-Wheeler transform is used in block-sorting compression (see [7]) and LZMA is a combination of dictionary compression with range coding that is often distributed in the XZ format [18]. These works define the candidate codecs, but are not concerned with the algorithm specifications discussed here, but with the algorithm itself.

B. Cloud Log Compression

Log data is highly structured because there are many lines corresponding to a recurring template of message structure. Drain parses logs online with a fixed size of tree [10] and Logzip does iterative clustering for better compression [11]. LogShrink uses regular expressions to recognize typical patterns and variations in the log messages [23] and Zhu et al. test automated log parsers with template-based methods [24]. CLP supports variables and templates, and can search compressed logs without decompression [12]. Loghub is a system log database for reproducible research [13] and system logs can be used for anomaly detection tasks [32]. The importance of structure-aware preprocessing is reflected in these works, but their tailored pipelines are predominantly geared towards log data, not general file upload and management pipelines.

C. Structured Cloud Activity Records

The most common cloud activity exports are CSV [17] and JSON [16]. Field names, duplicate schemas, and value distributions cause redundancies that are different from free-text logs. Analytical database systems use column-oriented storage, and there is some research on optimizing the design by compressing homogeneous columns, but it is unclear if such a rearrangement is applicable or valid for a given export.

D. AI-Assisted Compression and Algorithm Selection

Using trained networks to drive modelling is demonstrated in learned approaches to lossless compression, e.g. recurrent-network-based compressors [14] and neural-network compressors like NNCP [34] - typically at higher cost than dictionary-based approaches. Unlike conventional codec selection or analysis that is assisted by AI, this one helps select or analyze a method. The role of the AI Analysis component of the current system is explained in Section III-J.

E. File-Type Detection and Preprocessing

To reliably process heterogeneous uploads, it is necessary to validate the contents of the files and determine the type of the file before processing it in a specific way. For structured exports, there are format specifications for CSV [17] and JSON [16] which give rules to follow for parsing, and for

text logs, log parsing [10] offers a basis for template-oriented preprocessing of text logs. Alternatively, learned models have been used for content-type detection; for instance Magika is a deep-learning detector that has been shown to achieve an average F1 score of 99% on more than one million files on a test set that contains over a hundred content types [28]. It uses a simpler heuristic for the filename and content (Section III-C), which is not used in the present system.

F. Storage Optimization, Cost Estimation, and Integrity

The charges for object storage are based on the amount of storage, the storage class and region, and also on separate charges like requests, retrieval, and data transfer [15] as indicated in provider documentation. So, the effect of compression on storage cost is only relevant to the storage portion and for the stated price assumptions. To ensure integrity, cryptographic hashes like SHA-256 [9] can be used to compare the reconstructed file to the original, and to ensure reproducible benchmarking, fixed software versions, repeated timing and disclosed hardware are required, with careful summarization of results [25], [26].

G. Critical Synthesis and Research Gap

Codecs [1]–[8], log compressions [10]–[13], [23], and learned compression schemes [14] are thoroughly described by the reviewed references. What is less prominent in the reviewed works is a user-friendly pipeline combining categorization, preprocessing, actual compression, proven decompression, and cost estimate of storing the result. This work views its system as such a pipeline and is not intended to contribute any improvements to the algorithms behind. Table I summarizes representative works related to the problem.

III. Proposed Methodology

The proposed system is web based system for the management of files through compression. Supported files are uploaded, file details are reviewed, files are compressed and decompressed, and the size differences and estimated storage-cost impact of the reconstructed files are displayed, if supported. The system doesn't have to store log files; it's designed to work with any general file that's compatible with pipeline, and only to handle files of the two categories listed in Table II. The description of the web system in this section is based on a project source code, a TypeScript-based React front end and an Express-based back end. Implementation status for each component is given as implemented (which operates on the bytes of the file), simulated (which shows an estimated or fixed value without working with the bytes of the file), or advisory (which suggests an action that is not directly performed on the file bytes). All measurements presented in Section IV are from a separate benchmark harness, and are not from the web application.

A. System Architecture

It is designed in the architecture of presentation layer, application/processing layer and the data/metadata layer (Fig. 1). The presentation layer enables file uploads, file details, recent files, compression options, dashboard results (file count, file sizes, file space savings, and estimated cost savings), and decompression. The processing layer features: Validation, File-type Identification, File-type Categorization, Preprocessing Analysis, Compression, Measurement, Decompression, Integrity Checks, Cost Estimates, and AI Analysis. In the implementation, sections III-C through III-J explain the components that act directly on the bytes of a file.

The prototype is written in TypeScript. The presentation layer is created by using React 19 (19.2.8 locked), Vite 6 (6.4.3 locked), and Tailwind CSS 4, while JSZip and jsPDF are used for client-side archive and report exports, and Recharts for charts. The application layer uses a node.js Express 4 server (4.22.2 locked), started by using tsx. It exposes the `/api/health`, `/api/benchmark`, `/api/ai-suggestions` and `/api/ai-preprocess` to the end user both in development and production, serving Vite middleware to the built bundle. The locked versions are dependency versions, not runtime versions.

No database is used in the system. File records are stored in the in-memory state of React and are lost when the app reloads, except when it is demoed. The metadata, SHA-256 digest, line count and up to 20 non empty content lines are persisted with each record uploaded, while the original bytes and compressed outputs are not persisted. Only the demonstration session, theme, display settings and storage-threshold setting are stored in localStorage of the browser. Uploaded bytes are stored temporarily in the stateless back end until they are processed by a benchmark request; after processing, they are deleted. Therefore, the data layer keeps the files records and the sample content in memory but doesn't store original or compressed files.

The interface displays six groups of environments that you can use: AWS EC2, AWS Lambda, AWS S3, Azure Virtual Machines, Google Cloud Compute Engine, and Cloud Application Servers. These are folders in which files are grouped. The source has no AWS, Azure, or Google Cloud infrastructure SDK or API call – there is only an external service (Google Gemini API) accessed server side via the `@google/genai` package (2.21.0 locked) with the key provided by the `GEMINI_API_KEY` environment variable (Section III-J). This system is not said to be linked to real cloud infrastructure and no cloud integration is assessed in this paper.

The flow of the end-to-end processing is shown in Fig. 2. The file is uploaded, validated, the file-type is detected and categorized, appropriate pre-processing is applied, the compression engine creates the compressed file, sizes and times are recorded and shown on the dashboard, the user can later decompress the file and check for integrity if desired. This can be expressed as:

$$F \to V \to T \to K \to \Pi \to C \to M \to D_r \to I, \quad (1)$$

where F is uploaded file, V is validation, T is file-type detection, K categorization, Π is applicable preprocessing, C

TABLE I: Comparison of Related Work on Lossless Compression and Log Compression

S. No.	Author & Year	Title	Techniques Used	Pros	Cons
1	Ziv and Lempel (1977) [1]	A universal algorithm for sequential data compression	Sliding-window dictionary (LZ77)	Foundation of widely used dictionary codecs	Theoretical/algorithmic; no cloud-workflow evaluation
2	Huffman (1952) [2]	A method for the construction of minimum-redundancy codes	Prefix entropy coding	Optimal prefix codes for symbol frequencies	Symbol-level only; no repetition modelling
3	Deutsch (1996) [3], [4]	DEFLATE (RFC 1951) and GZIP (RFC 1952) specifications	LZ77 + Huffman; GZIP container with header and trailer	Ubiquitous, standardized, broadly supported	Specification-level; ratio limited versus newer codecs on some data
4	Collet and Kucherawy (2021) [5]	Zstandard Compression and the application/zstd Media Type (RFC 8878)	Zstandard format; levels; dictionaries	Standardized; tunable speed/ratio; dictionary support	Specification-level; behavior must be measured per dataset
5	Alakuijala and Szabadka (2016) [6]	Brotli Compressed Data Format (RFC 7932)	LZ77, Huffman, context modelling, static dictionary	Strong on text-like content	Specification-level; not benchmarked on cloud files here
6	Burrows and Wheeler (1994) [7]	A block-sorting lossless data compression algorithm	Block-sorting transform	Effective for text-like data	Block-based; run-time/memory trade-offs
7	Duda (2013) [8]	Asymmetric numeral systems	ANS entropy coding	Near-arithmetic efficiency at high speed	Entropy-coding component only
8	He et al. (2017) [10]	Drain: An online log parsing approach with fixed depth tree	Online template parsing	Efficient streaming parsing of logs	Parsing, not a full compression workflow

is compression, M is measurement, D_r is decompression, and I is optional integrity verification. When there is no category specific step, it is skipped (identity). The stages C and M and D_r and I are executed in the real file bytes only at the implemented web system; the other stages in the interface are simulated (Sections III-G to III-I).

B. Dataset and File Representation

Implementations include uploading each file as a record of its name, size, detected type, assigned type category, environment label, status, line count, and hashed SHA-256 of the original, plus some of the original lines and (after the simulated compression step) an estimate of the compressed size. Not stored in file record: Selected algorithm and level, compression and decompression time, verification status - returned by the benchmark endpoint per run. There are two categories that are kept separately (Table II). It is a mismatch to call all the activity records "log files", therefore the assignment of categories is stored explicitly.

This study uses 16 public Loghub log files and 8 synthetic activity files as a benchmark corpus [13]. The activity files are synthetic files and are not the actual cloud data, but both are described in Section IV-B. Future work could include standard compression corpora like [33] being added. This study was not able to use the project's initial test files (`ec2.log` and `ec2.csv`) and these are not included in the benchmark.

C. Automatic File-Type Detection

The file picker only validates the extension for files uploaded (narrowly, the extensions `.log`, `.txt`, `.json`, `.csv`, `.xml`, `.yml`, `.yaml`, `.gz`, `.tar` and `.tsv`), and does not re-validate it later; it does so via an HTML accept filter. The upload step does not check size and does not reject empty files: an empty or unreadable file is assigned a minimum of 512 B and several "place holder" sample lines. Size and emptiness checks are performed only in the benchmark endpoint, which rejects empty files, files larger than 10 MiB, and requests containing more than 20 files. All the functionality of file-type detection is in one function: filename extensions + content sniffing + filename keywords. If the name contains the words "audit" or "trail", or the first 1000 characters of the file begin with { or [then it is a Cloud Activity File: `cloudtrail` otherwise it is a `json-telemetry` Cloud Activity File. If the name contains the word "vpc" or "flow" the file is a Cloud Activity File: `vpc-flow` if it contains the words "kernel" or "audit" then it is a Cloud Activity File: `kernel-audit` otherwise it is an `apache-access` Cloud Activity File if the name contains the words "access", "http", "apache", or "nginx". Any other files, by default, are of type `syslog` (Cloud Log Files). Before saving, the detected category and type are populated in the drop-down fields that are editable. No signature-based or learned detection is used.

Content type JSON and CSV is not parsed and not checked against either [16] nor [17] - only the first character is checked with the JSON test. This is also a side-effect: if a file ends with 'csv' the category will be set to 'Cloud Log File' unless it has one of the above-mentioned key words in the name, or

TABLE II: File Categories Used in this Study

Category	Characteristics in this Study
Cloud Log Files	Textual, line-oriented system and application logs with timestamps and repetitive event templates. Evaluated using 16 Loghub samples (2,000 lines each) [13].
Cloud Activity Files	Structured tabular activity records (JSON/CSV) generated synthetically to emulate API call trails and audit event logs; evaluated using 8 synthetic files.

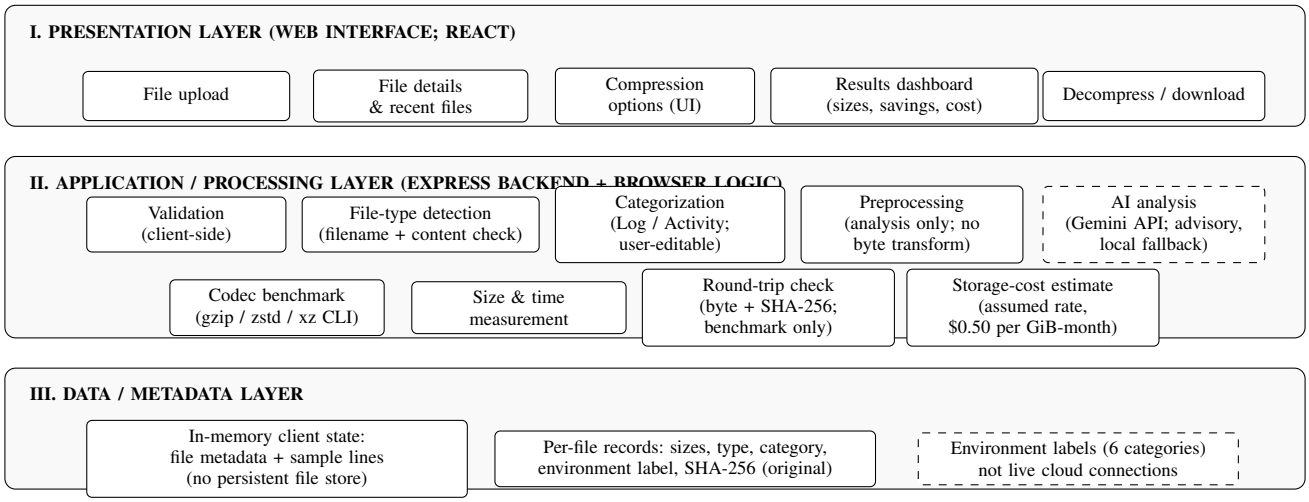


Fig. 1: Layered architecture of the implemented file-compression management system. Dashed elements indicate the advisory AI component and the organizational environment labels, which are not live integrations. Codec benchmarking and round-trip checking operate on file bytes only in the benchmark endpoint.

it is edited by the user. The unsupported or ambiguous files are not rejected, they use the default Cloud Log File path.

D. Cloud Log File Preprocessing

Cloud Log Files: No preprocessing to transform the bytes in the file is provided. The preprocessing module (`preprocessingEngine.ts` and the Preprocessing view) displays five types of analysis: Repeated patterns, Unnecessary whitespace and empty lines, Duplicate records, Common values, and File structure. These are calculated based on the record's size, number of lines, and type, with fixed multipliers (for instance, 18 percent of lines reported as repeated pattern, 1.5 percent of lines reported as empty line), instead of being derived from the content of the file, and the reported tokens, dictionary, and "NEPTS" stream are descriptive text. The server endpoint then passes the metadata of the file and up to three lines of the file to the AI model (Section III-J) and returns the JSON analysis, or a deterministic fallback constructed from the same fixed multipliers if the AI model is not available.

Only the fact that files explicitly marked as replicas or marked with file name markers like "_dup", "_replica", or "-copy" can be excluded from the selection passed to the compression step. In this sense possible methods as line normalisation and template based separation of constant and variable text (as studied in [10]–[12]) are proposed but not implemented. No reversibility statement is made for preprocessing as there is no byte-level transformation or inverse decoder, and the interface statements "preprocessing is verified reversible" are constants, not supported by an implemented reconstruction routine. Any pre-processing that is performed after this should be easily reversible, that is, actually the same as what was performed when the file was compressed, or the system should be able to notify the user that it does NOT compress byte exactly.

E. Cloud Activity File Preprocessing

In the case of Cloud Activity Files some forms of preprocessing were possible, such as a consistent ordering of records or grouping of records by field. None of these things are done: The system does not support CSV or JSON records, does not reorder or group fields, does not re-serialize records. The text in the interface on the columnar delta packing is explanatory only and the same analysis-only module as described in the previous subsection is used to process activity files. An activity-file transformation is therefore not applied and no reversibility is claimed. If parsing and re-serialization is added after [16], [17] then it should preserve the same number of bytes if there is a claim of integrity at the byte level.

F. Compression Algorithm Selection

The compression methods discussed in this paper—GZIP, ZSTD, and LZMA—are candidates. All three are not listed as selectable options in the web workflow. The single-file compression dialog doesn't provide any choice, but it is possible to compress using a guided pipeline with the choices Zstandard (preselected and marked "Recommended", which is described as level 19), LZ4 and Brotli.

Selection is thus manual and interface-driven. It is not rule based or model based: the algorithm is not selected by the AI part, and the weighted formulation in Eq. (10) is not carried out. The level is not a codec parameter, but rather the level of the interface. The benchmark endpoint applies GZIP, ZSTD, and LZMA (via gzip, zstd, and xz, respectively) using the default level for each tool; no level or parameter is revealed. The benchmark from Section IV was implemented in standard library code in a standalone harness, not by the web system's code. There's no one algorithm that is always best, depending on the objective, for instance, minimizing stored bytes versus minimizing runtime. The choice of weighted

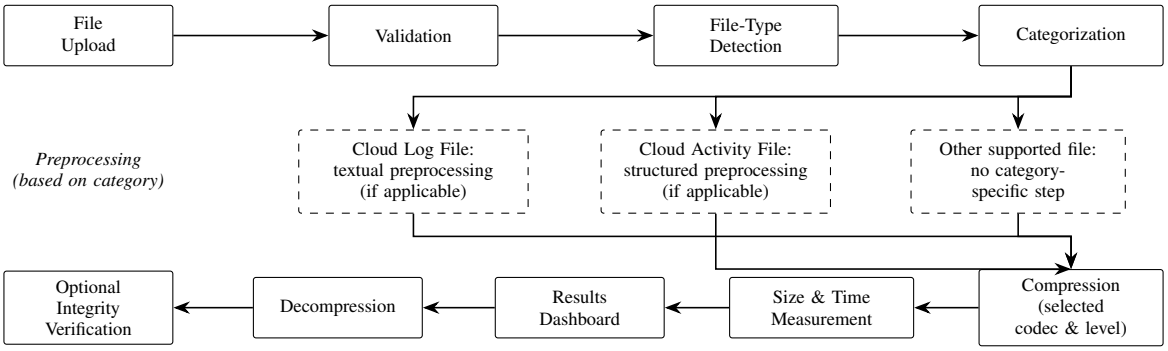


Fig. 2: Detailed processing flow from file upload to optional integrity verification. Category-specific preprocessing is applied only where applicable.

selection formulation in Eq. (10) is a possibility for such a choice and was not used in the reported benchmark.

G. Compression Engine

The designed compression engine reads and compresses the (optionally preprocessed) input, compresses the output, and also saves the compressed size and time. This behavior is only supported by the benchmark endpoint in the web system. The file is placed in the request body as base64 in a json document (request size limit 25 MB), written to a temporary file, and is compressed with the executable gzip, zstd or xz launched with the compressed bytes read from standard output (buffer size limit 20 MiB). Maximum allowed size: 10 MiB per file, 20 files per request. Whole-file processing (not streaming). Five runs are performed, one of which is discarded and the remaining five are measured; the median compression time, the size of the last run after compression, one decompression time and throughput based on the original size are returned. Timing is based around the process call using the Node.js monotonic clock (`process.hrtime`) which means that process start-up time is included but upload time and interface time (loading pages, etc.) is not included. The executables are those installed on the host, are not pinned in the source, and their versions are not recorded in the output, so no versions can be stated.

Although this engine is called from the compression actions in the dashboard, the single-file dialog, the batch dialog, and the guided pipeline do not call this engine. They impose a fixed size reduction (26% for single-file and batch, 24%, 26% and 32% for ZSTD, Brotli and LZ4 in the guided pipeline) and showcase a fake checksum, while making the progress display in the card more timely. Only the stored sample lines are used for downloads of "compressed" files. The values indicated for these flows are estimates and should not be interpreted as measured compression. The standalone benchmark harness (Section IV-A) adds the libraries listed in Table V to the web endpoint, runs 10 repetitions and includes 2 warm-ups.

H. Decompression and Integrity Verification

Decompression is the reverse operation to compression, in which the original file is recovered from the compressed representation, which involves inverting the compression step and any reversible pre-processing. In the web system, the digest of the original file is calculated in the browser at upload time, using the browser's Web Crypto API, and, if the file cannot be read, a placeholder digest is used. Only in the web application the decompression and verification is actually performed using the matching executable and the report of success depends on the bytes being restored to the original bytes as well as the digests being equal, using SHA-256 (Table V); the stand-alone harness in Section IV compares bytes and digests on its own. The interface actions of decompression and verification are simulated: The verification control displays a fixed-duration progress indicator and reports success without comparing anything, "restored" downloads are rebuilt from the stored sample lines, and the compressed digest is a generated place-holding. There is no requirement for verification in the workflow and the report needs to show if it has been done or not.

TABLE III: Candidate Compression Methods and Implementation Status

Method	**Role**	**Status (from source code)**
GZIP	DEFLATE-based [3], [4]; benchmarked (Sec. IV)	Implemented only in the benchmark endpoint (system gzip executable, default level) and in the browser `CompressionStream` used for sample-line downloads; not in the workflow selector
ZSTD	Zstandard [5]; benchmarked (Sec. IV)	Implemented only in the benchmark endpoint (system zstd executable, default level). The interface option is simulated: sizes are fixed-fraction estimates and exported `.zst` files carry a Zstandard magic number but hold uncompressed text
LZMA	LZMA/XZ [18]; benchmarked (Sec. IV)	Implemented only in the benchmark endpoint (xz executable, `-format=xz`); not in the workflow selector
Others	LZ4 and Brotli: not benchmarked	Offered in the guided pipeline selector but simulated only (fixed-fraction size estimates; no codec)

I. Storage-Cost Estimation

In the web system, the size in bytes is converted to an estimated cost as $(\text{bytes}/2^{30}) \times r$ where $1\ \text{GiB} = 2^{30} = 1,073,741,824$ bytes and the single constant rate is $r = \$0.50$ per GiB (the constant `CLOUD_PAYG_RATE_PER_GB` in the source, labelled "per GB", but applied to binary gibibytes), described there as a pay-as-you-use ingestion and storage rate for telemetry. Hence, throughout this paper this rate is assumed to be $p = \$0.50$ per GiB-month, where $G = 2^{30}$ bytes. An assumed rate set by the application, this is hard-coded without any price, provider, region, storage class, or date mentioned, and is not an actual cloud bill or official cloud-provider price. No mention of a billing period in the source, so it is assumed this paper uses the rate per month because of the cost perspectives of the application. Compressed size is provided to the estimator in the dashboard and is a fixed-fraction, not a measured size as described in Section III-G. The Cost and ROI view uses a ratio (default 4.8:1) that can be adjusted by the user and multiplied by 12 to get an annual number, and historical trend numbers are static data. The same rate is used in the illustration in Section IV-H and the observed aggregate ratios are those from the standalone benchmark. Real provider pricing is dependent on region, storage class, and also request, retrieval, and data-transfer charges [15]. The estimate, therefore, is just a storage estimate, assuming a certain rate, and does not outline the overall cost of the cloud bill.

J. AI Analysis Component

The AI Analysis component communicates with the Google Gemini API on the server with the use of the `@google/genai` client library. The code is requesting the model `gemini-2.5-flash` and if this is not available or a 503, it will try the same again using `gemini-2.5-flash-lite`, although in the interface it is worded as 'Gemini 3.8 Flash'. There is an `/api/ai-suggestions` endpoint that receives the user handwritten question, the metadata of the selected file (name, category, type, environment, sizes, status, line count, and up to five sample lines), and an overview of the total file count and volume. The model is asked to provide a JSON structured response with a direct answer, a prioritized list of suggestions with estimated impacts, a cost note considering the cost per GiB-month of the application's assumed price of $0.50, and a technical tip. The endpoint `/api/ai-preprocess` accepts metadata and up to 3 sample lines from each selected file and returns the suspected duplicate files in json format, a summary of the structural analysis, descriptive transformations and the estimated reduction percentage.

Without an API key, the call fails or the response is not parseable, the server returns a deterministic fallback, based on a keyword match on the question and fixed multipliers, with the source tagged on the response. The fallback texts include performance and saving numbers which were not measured in this study. The AI output is not used to select or use a compression algorithm and compression is not parameterized by nor dependent upon the AI output. One effect of its workflow is that it can exclude files that are suspected to be duplicates, which are detected by the preprocessing endpoint; this is the default behaviour in the guided pipeline. No training or evaluation procedure is given and this paper does not report accuracy of AI, the scope is explained in Section IV-I.

TABLE IV: Notation, Units, and Assumptions

Symbol	Meaning	Unit / Definition
S_o	Original file size	Bytes / MB
S_c	Compressed file size	Bytes / MB
CR	Compression ratio	S_o/S_c (dimensionless)
SS	Space saved	$(1 - S_c/S_o) \times 100\%$
T_c	Compression time	Seconds (median of repetitions)
T_d	Decompression time	Seconds (median of repetitions)
Θ_c	Compression speed	MB/s ($S_o/10^6/T_c$)
Θ_d	Decompression speed	MB/s ($S_o/10^6/T_d$)
V	Verification indicator	1 if byte-exact match, 0 otherwise
p	Assumed storage rate	\$0.50 per GiB-month (constant)
C_o, C_c	Storage cost	USD/month per 1,000 GiB
ΔC	Estimated savings	$C_o - C_c$ (USD/month)

K. Experimental Protocol

Each algorithm and each level is evaluated on each file in each category of files (Fig. 3). For each file/configuration combination, compression and decompression are run r times, with the median run time reported, along with the dispersion; the size of the compressed version of the file is recorded for each run (it was the same size each run, which is expected if the compression output is deterministic lossless). Round Trip verification is performed at the end of each repetition. The results are presented by category and codec. The versions of hardware, operating system, software and any caching conditions are revealed (Table V). Warm up runs are not included.

L. Mathematical Formulation

This section defines the evaluation measures. Sizes are in bytes, times in seconds, and measurements are made on the same file for each comparison. Notation is summarized in Table IV.

The compression ratio is the ratio of original to compressed size:

$$\text{CR} = S_o/S_c, \quad S_c > 0. \tag{2}$$

A value greater than 1 indicates reduction in size; a value below 1 indicates expansion. The percentage of space saved is

$$\text{SS} = (1 - S_c/S_o) \times 100\%. \tag{3}$$

For a set of N files, the aggregate ratio is $\text{CR}_{\text{agg}} = \sum S_{o,i} / \sum S_{c,i}$, which differs from the mean of per-file ratios and must be labeled accordingly. Compression time is the median of r repeated measurements,

$$T_c = \text{median}\{T_c^{(1)}, \ldots, T_c^{(r)}\}, \tag{4}$$

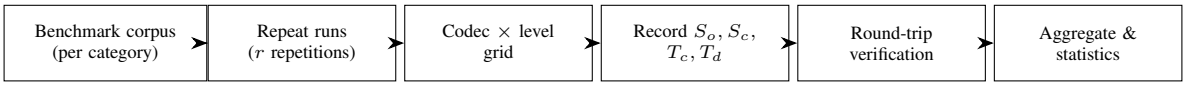


Fig. 3: Experimental protocol: repeated per-category benchmarking of codec and level combinations with round-trip verification and statistical aggregation.

and compression throughput is

$$\Theta_c = (S_o/10^6)/T_c \quad [\text{MB/s}]. \tag{5}$$

Decompression time T_d and throughput $\Theta_d = (S_o/10^6)/T_d$ are defined analogously on the original size so that the two are comparable. Integrity verification is defined as the indicator

$$V(F, F') = 1 \quad \text{if } H(F) = H(F'), \quad \text{else } 0. \tag{6}$$

Here F' is the file obtained by decompression (and reversal of any preprocessing). $V = 1$ supports byte-level equality up to the collision resistance of SHA-256 [9]; direct byte comparison may be used instead. When verification is not performed, the result is reported as not verified rather than as a pass.

Storage is measured in gibibytes, with G denoting the number of bytes in one GiB:

$$1 \text{ GiB} = G = 2^{30} \text{ bytes} = 1,073,741,824 \text{ bytes}. \tag{7}$$

The estimated monthly storage cost of the original and compressed files, for price p (currency per GiB-month), are

$$C_o = p \cdot (S_o/G), \quad C_c = p \cdot (S_c/G), \tag{8}$$

and the estimated storage-cost difference over m months is

$$\Delta C = m \cdot (C_o - C_c) = m \cdot p \cdot (S_o - S_c)/G. \tag{9}$$

A conservative net estimate subtracts any measurable compute cost c_{cpu} of compression: $\Delta C_{\text{net}} = \Delta C - c_{\text{cpu}}$. Assumptions: a single storage class and region; constant price p; no tiering, request, retrieval, or transfer charges; and no deletion or versioning effects. Because of these assumptions, ΔC is an estimate of the storage component only and is not the total cloud bill [15]. The values used in this study are stated in Section IV-H.

For algorithm selection, a weighted objective over a candidate set $\mathcal{A}$ can be used to formalize the ratio–time trade-off:

$$a^* = \arg\max_{a \in \mathcal{A}} [w_1 \cdot \text{SS}_a - w_2 \cdot T_{c,a} - w_3 \cdot M_a], \tag{10}$$

where M_a is peak memory (MB) and the terms should be normalized to comparable scales. This formulation was not used in the benchmark of Section IV and is not implemented in the web system, whose selection is manual (Section III-F).

Reported summary statistics use the sample mean and a t-based interval over n independent files,

$$\bar{x} \pm t_{\alpha/2, n-1} \cdot s/\sqrt{n}, \tag{11}$$

with s the sample standard deviation, and non-parametric tests where distributional assumptions are doubtful.

TABLE V: Experimental Configuration

Parameter	Value / Specification
Platform / CPU	1 vCPU, 3.9 GiB RAM (virtualized)
OS	Linux (Ubuntu 22.04 LTS kernel)
Python	3.10.12
gzip	System executable 1.10 (DEFLATE levels 1, 6, 9)
zstd	System executable 1.4.8 (levels 3, 10, 19)
xz	System executable 5.2.5 (LZMA levels 1, 6, 9)
Repetitions	10 per file and setting (1 warm-up discarded)
Timer	`time.perf_counter()`
Verification	SHA-256 byte-by-byte digest comparison

IV. Experimental Results and Analysis

In the following, the measurement coming out of the standalone benchmark harness described in Section IV-A are shown. Results are generated and represent the codecs that are tested on the given data and were not generated by the web application. AI Analysis is not assessed. Any and all reported values and raw timing measurements were taken from the benchmark harness for each repetition. The results are based on the documented protocol and have not been reproduced by others with the materials provided with this paper.

A. Experimental Configuration

Table V lists the configuration. Around each upload, each file was read into memory and compressed and decompressed in memory with a monotonic clock only, upload, disk persistence and user-interface time are not included. Two warm-up runs were thrown out for each file/codec setting, and ten runs were then timed and decompressed, with the median reported; all r runs gave the same output as the original. No measurements of peak memory were performed.

B. Dataset and File Characteristics

Two corpora were used (Table VI). Cloud Log Files: 16 public (the 2000 line samples of Apache, HDFS, OpenSSH, Linux, Hadoop, Spark, Zookeeper, BGL, Thunderbird, Android, HealthApp, HPC, Mac, OpenStack, Proxifier, and Windows logs) from the Loghub collection [13]. Cloud Activity Files: 8 synthetic files (4 CloudTrail-style JSON documents and 4 flat CSV exports of 2,000, 3,000, 5,000 and 8,000 records) generated using a fixed random seed by the generator script. They do not actually exist as cloud data; field names, event types and users are limited pools of data, while request and event identifiers are random. They are therefore only representative of exports with a high structural content, and may differ substantially from actual activity records.

TABLE VI: Dataset Characteristics

Category	Files	Total Size	Source / Types
Cloud Log	16	4.46 MB	Loghub 2,000-line samples (Android, Apache, BGL, HDFS, HPC, Hadoop, HealthApp, Linux, Mac, OpenSSH, OpenStack, Proxifier, Spark, Thunderbird, Windows, Zookeeper) [13]
Cloud Activity	8	12.51 MB	Synthetic generation: 4 CSV (370–1,480 KB), 4 JSON (1,019–4,079 KB) emulating API audit trails
Total	**24**	**16.97 MB**	**216 benchmark evaluations**

The log samples are small (151–595 KB). Ratios on small inputs are lower-bound indicators for large production logs, where compressors have more history to exploit.

C. Compression Performance

The aggregate compression ratio (Eq. 2 summed sizes) and the ratio–throughput balance for all nine codec settings are summarized in Fig. 4. In both cases higher ratios were achieved at the cost of lower compression throughput, and the three codec families fall into distinct areas of the trade-off.

D. Results for Cloud Log Files

In Table VII results are reported for the 16 log samples. Aggregate ratios ranged from 7.63 (GZIP-1) to 13.60 (LZMA-6 and LZMA-9), corresponding to space savings of 86.9% to 92.6%. The ratios are wide (e.g., 21.47 ± 3.91 for ZSTD-19), and are larger than aggregate ratios because the very compressible samples are equally important in the mean (Fig. 5). The aggregate ratio of ZSTD-3 at 548 MB/s was almost the same as that of GZIP-9 (40 MB/s), and was 13.6 times as high. The sizes of the compressed files were the same for LZMA-6 and LZMA-9, probably due to the fact that all entries are smaller than the 8 MiB dictionary used in preset-6.

E. Results for Cloud Activity Files

Table VIII shows the results for the 8 synthetic activity files. Aggregate ratios ranged from 4.94 (GZIP-1) to 7.23 (ZSTD-19), i.e., 79.8% to 86.2% space saved. The ordering of families is similar to the ordering of logs, with ratios smaller than for logs because of the random identifiers in these synthetic records which reduce the amount of redundancy. ZSTD-19 (7.23) and LZMA-6/9 (7.18) were close, while ZSTD-3 (5.96) approached GZIP-9 (6.08) at 15.8 times the throughput. Due to the synthetic nature of these data, results should not be used to draw conclusions about real cloud activity exports.

F. Algorithm Comparison

None of the settings performed best in all the criteria. In the runs on logs the highest ratio values were achieved by LZMA-6/9 (13.60) and ZSTD-19 (13.40), while for the activity files, ZSTD-19 achieved the highest ratio, but the lowest compression speed (1.9 MB/s) in the tests; in other runs LZMA-9 (3.8 MB/s) achieved the highest compression speed, but the lowest ratio (12.67) in the tests. In both types of compression (logs, 11.96 vs 9.99 at 80 MB/s versus 40 MB/s) ZSTD-10 outperformed GZIP-9. Also, ZSTD-3 was close to GZIP-9's ratio at higher speeds. This is a statement on these data and implementations and it is not a universal ranking. The throughput and the size views are presented in Fig. 6 and the statistical tests are in Section IV-J.

Trade-off between compression strength and time. The ratio increased from 9.96 to 13.40 (compressed output decreased from 10.0% to 7.5% of input size), and the compressed output decreased from 548 to 2.0 MB/s or 278 times slower. In the compression throughput, both the highest figures were for ZSTD-3 with 548 MB/s (logs) and 452 MB/s (activity files) and the highest ratios were for LZMA-6/9 (547 MB/s) on logs and ZSTD-19 (457 MB/s) on activity files. When both ratio and compression throughput are taken into account, it can be determined that the settings that are not dominated by the others are ZSTD-3, ZSTD-10, LZMA-6 for logs and ZSTD-3, ZSTD-10, ZSTD-19, LZMA-6 for activity files, with the latter being more preferred based on storage requirement and the former being more preferred based on speed requirement. Near ties in throughput are not robust, with the mean coefficient of variation of timing being 5–14%.

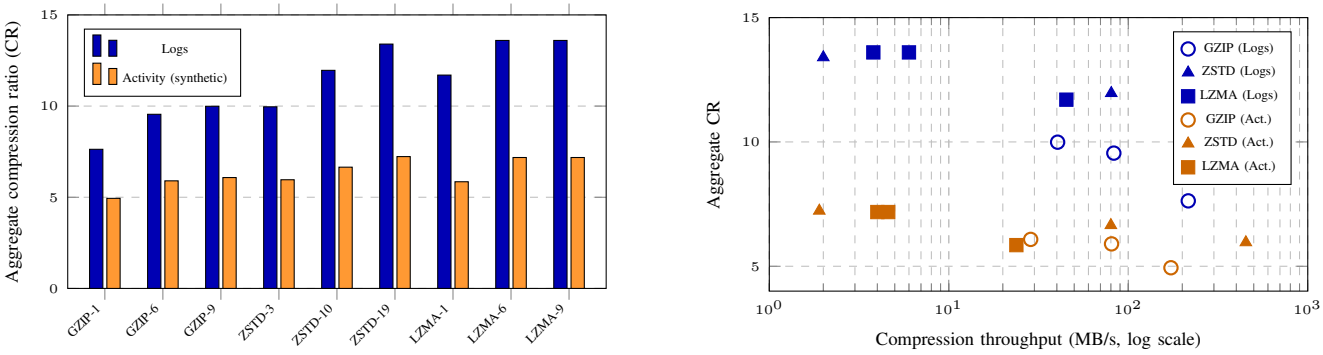
Public log samples versus synthetic activity files. In all of the settings, the aggregate ratio on the logs was higher than the aggregate ratio on the synthetic activity files by a factor of 1.54–2.00 (e.g., 13.40 vs. 7.23 for ZSTD-19). This difference is complicated by the fact that the two corpora are not of the same category (public vs. generated), size (151–595 KB vs. 370–4,079 KB), or format.

G. Decompression and Integrity Verification

With all 216 of 216 file–codec config used, the same 216 bytes were reproduced exactly as in the original file, i.e. $V = 1$ in Eq. (6) with 0 failures and no mismatches. The decompression ratios are all similar, but at a significantly different compression throughput, ranging from the 1.43–1.98 GB/s of Zstandard to the 95–187 MB/s of LZMA, representing an order of magnitude difference between families even when the ratios are close (Table IX).

H. Storage-Cost Impact

Equations (9) yield a storage cost for uncompressed data of $500.00 per month or a total of $6,000 over 12 months, which is the same as the annualized cost the application specifies. The rate is only an estimate that this web application uses, and the numbers generated are not a true price paid for a cloud bill; they are not an exact price quoted by the cloud provider. It thus applies the aggregate ratio per codec measured in this study to that volume; if we assume that the ratio is linear between codecs, then, assuming $C = 500$ per month for a codec, $\Delta C = 500 - C_c$ per month is the ratio per codec. This is an extrapolation from few files and synthetic activity data, and Fig. 7 compares the estimates. It is just for storage – it does not include cloud requests, retrieval, transfer, compute



(a) Compression ratio

(b) Ratio vs. throughput

Fig. 4: Compression performance on the benchmark corpus: (a) aggregate compression ratio by codec setting and category; (b) aggregate ratio versus compression throughput (logarithmic axis). Activity files are synthetic.

TABLE VII: Compression Results: Cloud Log Files, Standalone Benchmark (16 Public Loghub Samples)

Codec	Σ orig. (MB)	Σ comp. (MB)	Saved (MB)	CR (agg.)	CR mean $\pm$ 95% CI	SS (%)	Θ_c (MB/s)	CV_c (%)
GZIP-1	4.46	0.58	3.87	7.63	$9.03 $\pm$1.97$	86.9	215.9	9.2
GZIP-6	4.46	0.47	3.99	9.55	$11.45 $\pm$2.53$	89.5	83.1	6.5
GZIP-9	4.46	0.45	4.01	9.99	$12.23 $\pm$2.81$	90.0	40.4	8.1
ZSTD-3	4.46	0.45	4.01	9.96	$11.99 $\pm$2.86$	90.0	548.0	13.6
ZSTD-10	4.46	0.37	4.09	11.96	$14.75 $\pm$3.58$	91.6	80.4	10.8
ZSTD-19	4.46	0.33	4.13	13.40	$16.47 $\pm$3.91$	92.5	2.0	8.3
LZMA-1	4.46	0.38	4.08	11.70	$14.71 $\pm$3.59$	91.5	45.2	10.5
LZMA-6	4.46	0.33	4.13	13.60	$16.36 $\pm$3.71$	92.6	6.0	9.6
LZMA-9	4.46	0.33	4.13	13.60	$16.36 $\pm$3.71$	92.6	3.8	7.3

CR = compression ratio (Eq. 2); SS = space saved (Eq. 3); aggregate columns use summed sizes; $\Theta_c = \Sigma$ original MB$/\Sigma$ median compression time (Eq. 5); CV_c = median over files of the coefficient of variation of compression time across 10 repetitions.

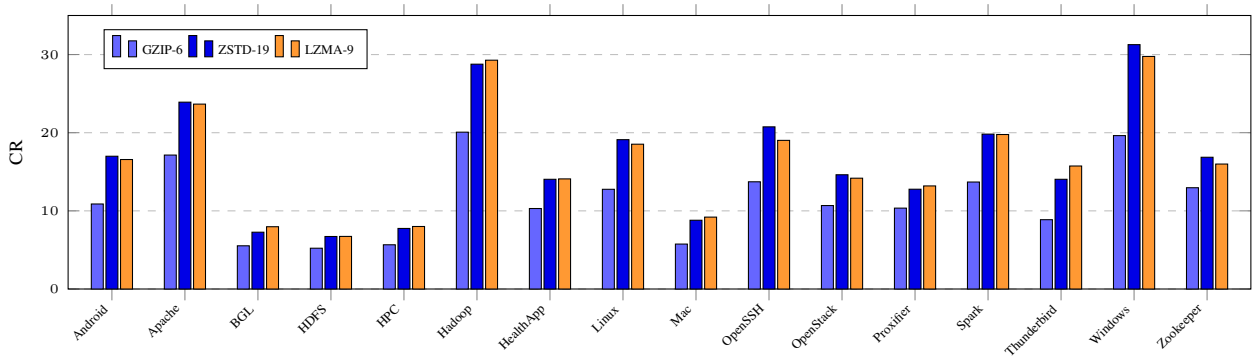


Fig. 5: Per-file compression ratio on the 16 Loghub samples for GZIP-6, ZSTD-19, and LZMA-9.

cost, etc. and it is not the overall cloud bill. Table X provides the detailed estimated monthly costs.

I. AI Component Evaluation

No evaluation of the AI Analysis component is reported. The component does not return a compression setting nor does it execute a compression setting, simply returns advisory, free text suggestions and a metadata level pre-processing summary (as described in Section III-J). There is no ground truth label associated with the output of the component in the benchmark. Thus, there is no study on accuracy, precision, or recommendation-quality metrics for it. An evaluation in the future could compare its suggested setting for each file to the best setting found by the benchmark of this section.

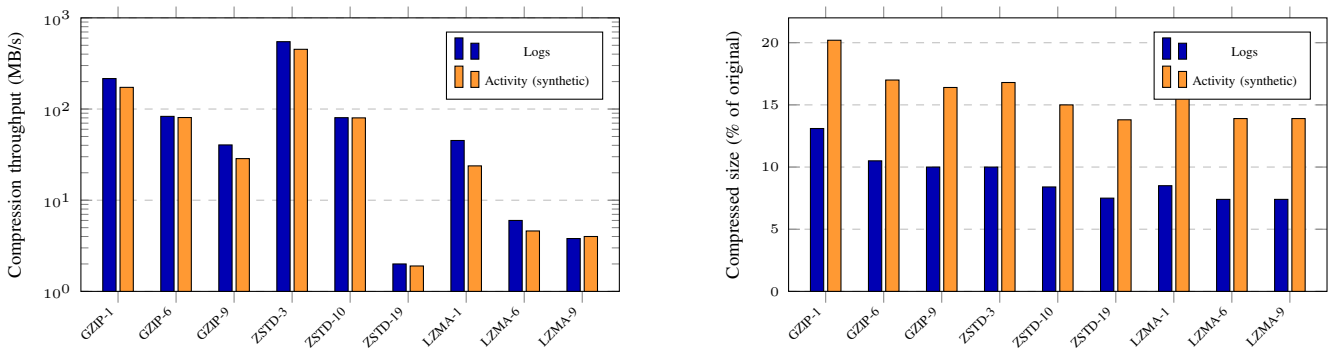
J. Statistical Analysis and Reproducibility

The intervals in Tables VII and VIII are t-based 95% intervals over files (see Eq. 11) with $n = 16$ and $n = 8$ respectively. Table XI shows two-sided Wilcoxon signed-rank tests [27] for per-file compression ratios of selected codec pairs: the last column lists the number of files for which the first codec achieved the better compression ratio. With $n = 8$ the smallest attainable two-sided p-value is 0.0078.

TABLE VIII: Compression Results: Cloud Activity Files, Standalone Benchmark (8 Synthetic Files)

Codec	Σ orig. (MB)	Σ comp. (MB)	Saved (MB)	CR (agg.)	CR mean $\pm$ 95% CI	SS (%)	Θ_c (MB/s)	CV_c (%)
GZIP-1	12.51	2.53	9.98	4.94	$4.76 $\pm$1.65$	79.8	172.9	4.9
GZIP-6	12.51	2.12	10.39	5.90	$5.76 $\pm$2.19$	83.0	80.7	13.0
GZIP-9	12.51	2.06	10.45	6.08	$5.98 $\pm$2.35$	83.6	28.6	5.0
ZSTD-3	12.51	2.10	10.41	5.96	$5.81 $\pm$2.13$	83.2	452.0	7.4
ZSTD-10	12.51	1.88	10.63	6.65	$6.57 $\pm$2.55$	85.0	80.0	9.1
ZSTD-19	12.51	1.73	10.78	7.23	$7.21 $\pm$2.99$	86.2	1.9	6.7
LZMA-1	12.51	2.14	10.37	5.85	$5.69 $\pm$2.12$	82.9	23.8	11.5
LZMA-6	12.51	1.74	10.76	7.18	$7.04 $\pm$2.71$	86.1	4.6	7.9
LZMA-9	12.51	1.74	10.76	7.18	$7.04 $\pm$2.71$	86.1	4.0	9.3

Synthetic data generated for this study; not real cloud records. Column definitions as in Table VII.



(a) Compression throughput

(b) Compressed vs. original size

Fig. 6: Compression throughput (logarithmic axis) and compressed size as a percentage of original size, by codec setting and category. Activity files are synthetic.

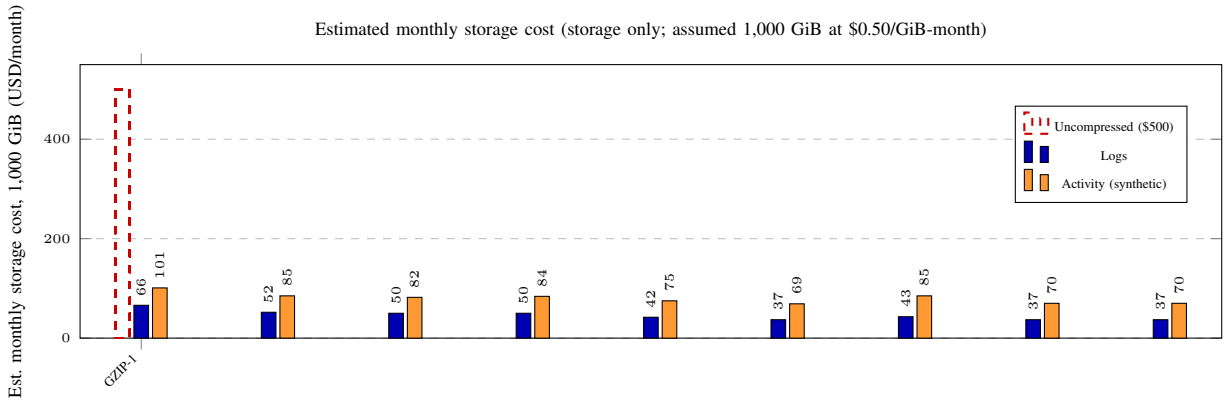


Fig. 7: Estimated monthly storage cost for 1,000 GiB of data at the application's assumed rate of $0.50 per GiB-month. The dashed line represents the uncompressed estimate of $500 per month. Storage-only estimates are extrapolated from measured compression ratios and are not provider prices or billing records.

There was no significant difference at the level of significance $\alpha = 0.05$ between ZSTD-19 and LZMA-9, as well as between ZSTD-3 and GZIP-6. The median coefficient of variation of compression time per file for logs ranged from 6.5% to 13.6%; and activity files ranged from 4.9% to 13.0%; of this variability only small differences should be over-interpreted. No multiple comparisons were made and the sample of activities is small and synthetic; thus the activity tests are to be interpreted descriptively.

Table XII shows the per-file ratios of the various settings. Assumptions: that each test is applied to a pair of settings within the same file, and that each file can be considered as a single observation. This is possible for the 16 logs, which were created by different systems, but not for the 8 activity files, created by one generator; those tests are descriptive. The Wilcoxon test does not require the ratios to be normally distributed, while the t-based intervals make the assumption of approximately normal ratios per file, which is unlikely for

TABLE IX: Decompression Throughput and Verification

Codec	Θ_d logs (MB/s)	Θ_d activity (MB/s)	Verified / total	Failed
GZIP-1	488	444	24 / 24	0
GZIP-6	569	458	24 / 24	0
GZIP-9	564	468	24 / 24	0
ZSTD-3	1595	1921	24 / 24	0
ZSTD-10	1659	1850	24 / 24	0
ZSTD-19	1428	1985	24 / 24	0
LZMA-1	187	123	24 / 24	0
LZMA-6	178	104	24 / 24	0
LZMA-9	171	95	24 / 24	0

TABLE X: Estimated Monthly Storage Cost per 1,000 GiB (Storage Only; Assumed Application Rate of $0.50 per GiB-Month)

Codec	Logs C_c ($)	Logs ΔC ($)	Activity C_c ($)	Activity ΔC ($)
GZIP-1	65.53	434.47	101.21	398.79
GZIP-6	52.36	447.64	84.75	415.25
GZIP-9	50.05	449.95	82.24	417.76
ZSTD-3	50.20	449.80	83.89	416.11
ZSTD-10	41.81	458.19	75.19	424.81
ZSTD-19	37.31	462.69	69.16	430.84
LZMA-1	42.74	457.26	85.47	414.53
LZMA-6	36.76	463.24	69.64	430.36
LZMA-9	36.76	463.24	69.64	430.36

Basis: the application's assumed rate of $0.50 per GiB-month (`CLOUD_PAYG_RATE_PER_GB`), 1 GiB $= 2^{30}$ bytes, assumed volume 1,000 GiB (uncompressed: $500.00 per month), $C_c = 500/\mathrm{CR}_{\mathrm{agg}}$ from Tables VII and VIII. This is an assumed rate, not an official provider price, and the figures are estimates, not billing records.

TABLE XI: Paired Wilcoxon Signed-Rank Tests on Per-File Compression Ratio

Category	Comparison (A vs B)	W	p-value	A higher in
Logs	ZSTD-19 vs LZMA-9	58	0.632	8/16
Logs	ZSTD-19 vs GZIP-9	0	3.1e-5	16/16
Logs	LZMA-9 vs GZIP-9	0	3.1e-5	16/16
Logs	ZSTD-10 vs GZIP-9	0	3.1e-5	16/16
Logs	ZSTD-3 vs GZIP-6	45	0.252	9/16
Activity	ZSTD-19 vs LZMA-9	10	0.313	4/8
Activity	ZSTD-19 vs GZIP-9	0	0.008	8/8
Activity	LZMA-9 vs GZIP-9	0	0.008	8/8
Activity	ZSTD-10 vs GZIP-9	0	0.008	8/8
Activity	ZSTD-3 vs GZIP-6	8	0.195	6/8

8–16 skewed ratios, and therefore are only indicative.

The reported measurements are done according to the documented protocol (Sections III-K and IV-A) and Table V (corpora, codec libraries and versions, levels, warm-up and repetition count, timer and integrity check). The web-application source code supplied with this study does not include the generator, benchmark, analysis, and audit scripts and the raw per-run timing files of the standalone benchmark; there is no public repository or DOI available for these. The benchmark is therefore not included as a complete reproduction package: The exact materials and numbers are not provided here and the web application's own `/api/benchmark` endpoint (Section III-G) follows a different procedure and cannot regenerate the materials and numbers. Tables VII–XII report the aggregate and per-file results. Samples for the Loghub are taken from the public repository [13]. The results vary based on hardware and library versions, and a single vCPU was utilized.

V. Discussion

The measurements illustrate how ratio and speed trade off and reveal that the trade-off varies between the codec families. For log samples, higher Zstandard and LZMA settings gave aggregate ratios of approximately 13.4–13.6 compared to 10.0 for the highest compression throughput setting of GZIP, with LZMA still being relatively quick to decompress and Zstandard still being quite fast. The results are consistent in direction with the results in the literature with respect to general-purpose codecs [3]–[8], but do not compare favourably with log-specific compressors [10]–[12], [23], which were not evaluated here and whose results are reported on significantly larger log corpora.

The results of compression were not as good on the synthetic activity files as on the logs, demonstrating that the compression ratio that can be obtained is dependent on the redundancy of the file's content; random identifiers are expensive for any lossless compression method. The absolute saving from the same ratio only comes into play for larger storage volume, and the value of this ratio varies based on the storage class, region, retention period, and price (Table X) and excludes request, retrieval, transfer, and compute charges. There is no algorithm which is always "the best" — it is dependent on what is most important: the number of bytes stored, the compression time or the decompression time.

These conclusions were drawn from the practical constraints. The inputs for log data were small samples, activity data were synthetic, a single vCPU machine was used, peak memory was not measured, the benchmark was a test of implementations in libraries instead of the actual web application deployed, there was no log-specific preprocessing evaluated and the six categories of environments were organizational labels, with no verified cloud integrations, and the AI Analysis component was not evaluated. Furthermore, the analysis of the web app's source reveals the following: compression, decompression, verification, and cost figures of the dashboard are simulated or assumed (Sections III-G to III-I) instead of real file bytes; no byte-level preprocessing is done; the detection of file types is based on heuristics on the filename and on the first character of the file; files are stored only as metadata and as a sample of lines read from it; only the endpoint of the benchmark operates on real file bytes.

A. Comparison with Reference Work

We compare against Wei et al. [35] (USENIX FAST 2021), *On the Feasibility of Parser-based Log Compression in Large-Scale Cloud Systems*, chosen because it measures log compression at cloud scale. It evaluates log files only: gzip, LZMA, LogArchive, Logzip and its own parser-based LogReducer (templates, delta timestamps, correlation, elastic encoding, LZMA packing) on 18 AliCloud production and 16 public log types, whereas this work benchmarks GZIP, Zstandard and LZMA, without log-specific preprocessing, on both Cloud Log Files and Cloud Activity Files. Metrics are original and compressed size, compression ratio, storage saved, and speed

TABLE XII: Per-File Compression Ratios for All Nine Settings (CR = Original / Compressed Size)

File	Size (KB)	GZIP-1	GZIP-6	GZIP-9	ZSTD-3	ZSTD-10	ZSTD-19	LZMA-1	LZMA-6	LZMA-9
Android (log)	279.1	8.41	10.88	11.24	12.89	15.48	16.99	15.08	16.57	16.57
Apache (log)	171.2	12.55	17.14	18.65	15.85	19.73	23.91	20.14	23.66	23.66
BGL (log)	317.1	4.32	5.53	5.59	5.12	6.30	7.27	6.34	7.97	7.97
HDFS (log)	287.8	4.41	5.22	5.34	5.32	5.94	6.72	5.53	6.73	6.73
HPC (log)	151.2	4.77	5.66	5.90	5.40	6.47	7.75	6.45	8.00	8.00
Hadoop (log)	384.9	16.56	20.07	21.18	22.22	26.44	28.77	27.44	29.28	29.28
HealthApp (log)	187.5	7.92	10.30	10.98	9.73	12.30	14.04	12.81	14.09	14.09
Linux (log)	216.5	10.52	12.76	14.68	14.37	17.88	19.11	18.19	18.53	18.53
Mac (log)	319.4	4.61	5.75	5.87	7.06	8.26	8.80	7.96	9.20	9.20
OpenSSH (log)	225.2	11.09	13.72	14.48	13.50	17.87	20.75	17.99	19.02	19.02
OpenStack (log)	595.1	8.40	10.68	11.29	11.61	13.38	14.62	11.72	14.18	14.18
Proxifier (log)	237.0	7.78	10.35	10.83	9.31	11.77	12.77	11.29	13.19	13.19
Spark (log)	196.3	10.00	13.69	15.03	13.95	17.64	19.82	18.22	19.77	19.77
Thunderbird (log)	325.2	7.62	8.87	9.26	10.63	11.96	14.04	13.76	15.74	15.74
Windows (log)	285.4	15.45	19.63	21.53	23.30	28.93	31.27	27.41	29.76	29.76
Zookeeper (log)	279.9	10.01	12.96	13.91	11.64	15.60	16.86	14.96	15.99	15.99
activity_csv_1.csv (synthetic)	369.9	2.91	3.30	3.35	3.44	3.73	3.88	3.30	3.99	3.99
activity_csv_2.csv (synthetic)	555.3	2.92	3.31	3.35	3.43	3.73	3.90	3.31	4.02	4.02
activity_csv_3.csv (synthetic)	924.5	2.92	3.31	3.35	3.42	3.71	3.82	3.32	4.05	4.05
activity_csv_4.csv (synthetic)	1480.4	2.92	3.32	3.36	3.41	3.68	3.86	3.34	3.97	3.97
activity_json_1.json (synthetic)	1019.4	6.58	8.19	8.59	8.25	9.55	10.41	7.98	10.06	10.06
activity_json_2.json (synthetic)	1529.6	6.61	8.21	8.61	8.22	9.49	10.52	8.05	10.07	10.07
activity_json_3.json (synthetic)	2548.4	6.61	8.22	8.63	8.16	9.39	10.60	8.09	10.09	10.09
activity_json_4.json (synthetic)	4078.6	6.61	8.25	8.64	8.13	9.26	10.66	8.12	10.09	10.09

Each value is the ratio for one file and one setting; these are the per-file values behind the aggregate results in Tables VII and VIII and the tests in Table XI. Activity files are synthetic.

and time; this work adds decompression throughput and round-trip verification (216/216 byte-exact), which [35] does not report. The ratio ranges are not like-for-like, given the different data, hardware and scale. Table XIII summarizes both studies using only reported values.

VI. Conclusion and Future Work

This paper introduced the concept of a categorized file-compression management workflow for two types of cloud files, Cloud Log Files and Cloud Activity Files, formalized its evaluation measures, and reported a documented benchmark of three compression levels for each compression method: GZIP, Zstandard, and LZMA. All 216 round trips were byte-exact on 16 public Loghub log samples and 8 synthetic activity files; Zstandard with level 3 compression gave the best compression (between 7.63 and 13.60 on logs and 4.94 and 7.23 on the synthetic activity files); Zstandard was the fastest to decompress overall, and the fastest to compress on the synthetic activity files. The results herein are for these specific pieces of data and not a comprehensive end-to-end analysis of the web application.

Extensions of the current work are the connection of the actions of compression, decompression and verification of the web workflow to the actual codec engine, and the evaluation end-to-end of the deployed system, as well as the extension and evaluation of the preprocessing of logs and activity files in a reversible way, the use of larger real logs corpora and real activity files, the determination of the memory usage, the comparison of the log-specific compressors, the verification of the integration with the cloud, the evaluation of the AI Analysis component, the replacement of the assumed storage rate with documented provider price, and the extension of the support for file formats.

TABLE XIII: Comparison with Wei et al. [35] (as Reported)

Metric	This work	Wei et al. [35]	Better / note
Methods	GZIP 1/6/9, ZSTD 3/10/19, LZMA 1/6/9; no log-specific step	gzip, LZMA, LogArchive, Logzip vs, LogReducer (parser-based + LZMA)	Different goals; [35] stronger method
Dataset	16 Loghub logs (2,000 lines) + 8 synthetic activity; 24 files	18 AliCloud production + 16 public log types	[35]: scale, real data
Original size	Logs 4.46 MB; activity 12.51 MB	1.76 TB production; 77 GB public	[35]: scale
Compressed size	Logs 0.33–0.58 MB; activity 1.73–2.53 MB	LogReducer 34.25 GB; gzip 152.03; LZMA 107.22; LogArchive 91.54; Logzip 89.86 GB	Not comparable
Compression ratio	Logs 7.63–13.60; activity 4.94–7.23 (aggregate)	Production: LogReducer 13.67–109.51, LZMA 8.25–22.82 (per log)	[35] (different data)
Storage saved	Logs 86.9–92.6%; activity 79.8–86.2%	LogReducer 98.10% (100 – 1.90%, derived); others Not Reported	[35] (different data)
Compression speed	Logs 2.0–548.0; activity 1.9–452.0 MB/s	LogReducer 3.77–10.72; LZMA 2.48–8.22 MB/s (production)	This work faster; not comparable
Decompression speed	Logs 171–1659; activity 95–1985 MB/s	Not Reported	This work only
Execution time	Median of 10 runs; CV 4.9–13.6%; total time Not Reported	1.76 TB: LogReducer 58.19 h; gzip 25.35 h; LZMA 91.54 h	Different scope
Integrity check	216/216 round trips byte-exact (SHA-256)	Not Reported	This work only
Test setup	1 vCPU, 3.9 GiB, Python; peak memory Not Reported	4 servers, 16 cores each, 188 GB, 4 threads; C/C++	Not comparable

References

[1] J. Ziv and A. Lempel, "A universal algorithm for sequential data compression," *IEEE Trans. Inf. Theory*, vol. 23, no. 3, pp. 337–343, May 1977.

[2] D. A. Huffman, "A method for the construction of minimum-redundancy codes," *Proc. IRE*, vol. 40, no. 9, pp. 1098–1101, Sep. 1952.

[3] P. Deutsch, "DEFLATE compressed data format specification version 1.3," RFC 1951, IETF, May 1996.

[4] P. Deutsch, "GZIP file format specification version 4.3," RFC 1952, IETF, May 1996.

[5] Y. Collet and M. Kucherawy, "Zstandard compression and the 'application/zstd' media type," RFC 8878, IETF, Feb. 2021.

[6] J. Alakuijala and Z. Szabadka, "Brotli compressed data format," RFC 7932, IETF, Jul. 2016.

[7] M. Burrows and D. J. Wheeler, "A block-sorting lossless data compression algorithm," Digital Equipment Corp. Systems Research Center, Res. Rep. 124, May 1994.

[8] J. Duda, "Asymmetric numeral systems: Entropy coding combining speed of Huffman coding with compression rate of arithmetic coding," arXiv:1311.2540, 2013.

[9] National Institute of Standards and Technology, "Secure hash standard (SHS)," FIPS PUB 180-4, Aug. 2015.

[10] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, 2017, pp. 33–40.

[11] J. Liu, J. Zhu, S. He, P. He, Z. Zheng, and M. R. Lyu, "Logzip: Extracting hidden structures via iterative clustering for log compression," in *Proc. IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2019, pp. 863–873.

[12] K. Rodrigues, Y. Luo, and D. Yuan, "CLP: Efficient and scalable search on compressed text logs," in *Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2021, pp. 183–198.

[13] S. He, J. Zhu, P. He, and M. R. Lyu, "Loghub: A large collection of system log datasets towards automated log analytics," arXiv:2008.06448v1, 2020.

[14] M. Goyal, K. Tatwawadi, S. Chandak, and I. Ochoa, "DeepZip: Lossless data compression using recurrent neural networks," arXiv:1811.08162, 2018.

[15] Amazon Web Services, "Amazon S3 pricing," https://aws.amazon.com/s3/pricing/ (accessed Oct. 18, 2026).

[16] T. Bray, Ed., "The JavaScript Object Notation (JSON) data interchange format," RFC 8259, IETF, Dec. 2017.

[17] Y. Shafranovich, "Common format and MIME type for comma-separated values (CSV) files," RFC 4180, IETF, Oct. 2005.

[18] L. Collin and I. Pavlov, "The .xz file format," version 1.2.1, Apr. 8, 2024. [Online]. Available: https://tukaani.org/xz/xz-file-format.txt

[19] C. E. Shannon, "A mathematical theory of communication," *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, Jul. 1948.

[20] J. Ziv and A. Lempel, "Compression of individual sequences via variable-rate coding," *IEEE Trans. Inf. Theory*, vol. 24, no. 5, pp. 530–536, Sep. 1978.

[21] T. A. Welch, "A technique for high-performance data compression," *Computer*, vol. 17, no. 6, pp. 8–19, Jun. 1984.

[22] I. H. Witten, R. M. Neal, and J. G. Cleary, "Arithmetic coding for data compression," *Commun. ACM*, vol. 30, no. 6, pp. 520–540, Jun. 1987.

[23] X. Li, H. Zhang, V.-H. Le, and P. Chen, "LogShrink: Effective log compression by leveraging commonality and variability of log data," in *Proc. IEEE/ACM 46th Int. Conf. Software Engineering (ICSE)*, 2024, pp. 23:1–23:12, doi: 10.1145/3597503.3608129.

[24] J. Zhu, S. He, J. Liu, P. He, Q. Xie, Z. Zheng, and M. R. Lyu, "Tools and benchmarks for automated log parsing," in *Proc. IEEE/ACM 41st Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019, pp. 121–130.

[25] P. J. Fleming and J. J. Wallace, "How not to lie with statistics: The correct way to summarize benchmark results," *Commun. ACM*, vol. 29, no. 3, pp. 218–221, Mar. 1986.

[26] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*, 2015, Art. no. 73.

[27] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, Dec. 1945.

[28] Y. Fratantonio et al., "Magika: AI-powered content-type detection," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2025, arXiv:2409.13768.

[29] M. Armbrust et al., "A view of cloud computing," *Commun. ACM*, vol. 53, no. 4, pp. 50–58, Apr. 2010.

[30] P. Mell and T. Grance, "The NIST definition of cloud computing," NIST Special Publication 800-145, Sep. 2011.

[31] K. Kent and M. Souppaya, "Guide to computer security log management," NIST Special Publication 800-92, Sep. 2006.

[32] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience report: System log analysis for anomaly detection," in *Proc. IEEE 27th Int. Symp. Software Reliability Engineering (ISSRE)*, 2016, pp. 207–218.

[33] R. Arnold and T. Bell, "A corpus for the evaluation of lossless compression algorithms," in *Proc. Data Compression Conf. (DCC)*, 1997, pp. 201–210.

[34] F. Bellard, "Lossless data compression with neural networks," Mar. 30, 2019. [Online]. Available: https://bellard.org/nncp/nncp.pdf

[35] J. Wei, G. Zhang, Y. Wang, Z. Liu, Z. Zhu, J. Chen, T. Sun, and Q. Zhou, "On the feasibility of parser-based log compression in large-scale cloud systems," in *Proc. USENIX Conf. File and Storage Technologies (FAST)*, 2021, pp. 249–262.